

Relatório da Atividade 3

Compressão, Representação e Descrição

Layza Maria Rodrigues Carneiro

22 de maio de 2026

Disciplina: Processamento de Imagens

Professor: Matheus Araújo

1 Introdução

Esta atividade teve como objetivo explorar e consolidar conceitos de compressão, representação e descrição de imagens digitais por meio da implementação prática de técnicas fundamentais, como transformações, redução de redundâncias e extração de características. A proposta buscou desenvolver a compreensão dos princípios que permitem representar os dados visuais de forma eficiente e descrever quantitativamente o conteúdo das imagens.

Seguindo as orientações da atividade, as operações principais foram desenvolvidas sem o uso de funções prontas para realizar diretamente os processamentos solicitados. As imagens utilizadas pertencem ao tema "Animais e Natureza".

2 Metodologia

O desenvolvimento foi realizado em Python, com apoio das bibliotecas OpenCV, NumPy e Matplotlib. O OpenCV foi empregado principalmente para leitura e salvamento das imagens, enquanto as etapas centrais de processamento, representação, compressão e extração de características foram implementadas manualmente.

3 Link do Projeto

O código-fonte desenvolvido para a atividade foi disponibilizado no repositório:

https://github.com/LayzaCarneiro/ProcessamentoDeImagens/tree/main/atividade_tres

Observação: o projeto também foi entregue em arquivo .zip.

4 Resultados e Análises

4.1 Questão 1 – Compressão de imagem com DCT

Descrição

Na primeira questão, foi implementado um processo de compressão de imagem inspirado no funcionamento do JPEG, utilizando a Transformada Discreta do Cosseno (DCT). A imagem foi convertida para escala de cinza, dividida em blocos quadrados e processada individualmente por meio da DCT bidimensional, seguida pelas etapas de quantização, desquantização e reconstrução com a transformada inversa (IDCT).

O objetivo principal foi observar como a representação da imagem no domínio da frequência permite concentrar grande parte da informação visual em poucos coeficientes, possibilitando a redução de redundâncias. Além disso, o código foi adaptado para trabalhar com diferentes tamanhos de blocos, permitindo a realização de testes com blocos de 2x2, 8x8, 16x16 e 64x64. Essa flexibilidade tornou possível comparar o impacto do tamanho dos blocos na qualidade da imagem reconstruída e na percepção das perdas causadas pela quantização.

Implementação

O código foi organizado nas seguintes etapas principais:

- definição do tamanho dos blocos utilizados no processamento;
- conversão manual da imagem colorida para escala de cinza;
- implementação manual da DCT bidimensional para cada bloco;
- implementação manual da IDCT para reconstrução dos blocos comprimidos;
- adaptação dinâmica da matriz de quantização conforme o tamanho do bloco;
- divisão da imagem em blocos NxN, aplicação da compressão e reconstrução final;
- salvamento da imagem original em cinza, da imagem reconstruída, da imagem de diferença e do comparativo visual.

Um ponto importante da implementação foi o uso da variável `BLOCK_SIZE`, que controla o tamanho dos blocos usados na DCT e na IDCT. Com isso, o mesmo código pôde ser executado para diferentes configurações, como 2x2, 8x8, 16x16 e 64x64, sem alterar a estrutura principal do algoritmo.

Listing 1: Definição do tamanho dos blocos usados na compressão.

```
# Tamanho dos blocos usados na DCT/IDCT
# Ajuste para 2, 8, 16, 64, etc.
BLOCK_SIZE = 64
```

Antes da compressão, a imagem foi convertida manualmente para tons de cinza, utilizando a fórmula de luminância ponderada dos canais RGB. Essa etapa garante que o processamento seja realizado sobre uma única matriz de intensidades.

Listing 2: Conversão manual da imagem colorida para escala de cinza.

```
def to_gray(img):
    altura, largura, _ = img.shape
    gray = np.zeros((altura, largura), dtype=np.uint8)

    for y in range(altura):
        for x in range(largura):
            b = img[y, x, 0]
            g = img[y, x, 1]
            r = img[y, x, 2]

            valor = int(0.299 * r + 0.587 * g + 0.114 * b)
            gray[y, x] = min(valor, 255)

    return gray
```

A DCT bidimensional foi implementada manualmente, sem o uso de funções prontas para a transformação. Em cada bloco, os valores dos pixels foram centralizados em torno de zero por meio da subtração de 128, procedimento comum em esquemas baseados no JPEG. Em seguida, os coeficientes foram calculados por somatórios envolvendo funções cosseno e constantes de normalização.

Listing 3: Implementação manual da DCT bidimensional em blocos NxN.

```
def dct2d(bloco, N=BLOCK_SIZE):
    dct_bloco = np.zeros((N, N))
    bloco_f = bloco.astype(float) - 128.0

    for u in range(N):
        for v in range(N):
            soma = 0.0
            for x in range(N):
                for y in range(N):
                    cos_x = math.cos(((2*x + 1) * u * math.pi) / (2 * N))
                    cos_y = math.cos(((2*y + 1) * v * math.pi) / (2 * N))
                    soma += bloco_f[x, y] * cos_x * cos_y

            cu = math.sqrt(1/N) if u == 0 else math.sqrt(2/N)
            cv = math.sqrt(1/N) if v == 0 else math.sqrt(2/N)
            dct_bloco[u, v] = cu * cv * soma

    return dct_bloco
```

Para reconstruir a imagem após a quantização, foi implementada a IDCT bidimensional. Essa etapa realiza o processo inverso, retornando os blocos do domínio da frequência para o domínio espacial e somando novamente 128 para desfazer a centralização inicial.

Listing 4: Reconstrução dos blocos por meio da IDCT manual.

```
def idct2d(bloco_dct, N=BLOCK_SIZE):
    bloco_reconstruido = np.zeros((N, N))

    for x in range(N):
        for y in range(N):
```

```

soma = 0.0
for u in range(N):
    for v in range(N):
        cu = math.sqrt(1/N) if u == 0 else math.sqrt(2/N)
        cv = math.sqrt(1/N) if v == 0 else math.sqrt(2/N)
        cos_x = math.cos(((2*x + 1) * u * math.pi) / (2 * N))
        cos_y = math.cos(((2*y + 1) * v * math.pi) / (2 * N))
        soma += cu * cv * bloco_dct[u, v] * cos_x * cos_y

bloco_reconstruido[x, y] = soma + 128.0

return bloco_reconstruido

```

Outro trecho relevante foi a adaptação da matriz de quantização. Como a matriz base possui dimensão 8x8, foi criada uma função para ajustá-la dinamicamente ao tamanho de bloco escolhido. Para blocos maiores múltiplos de 8, a matriz é expandida com o produto de Kronecker; para outros tamanhos, é utilizada uma matriz uniforme. Isso permitiu executar os testes com diferentes dimensões de bloco mantendo o fluxo geral da compressão.

A `MATRIZ_Q_BASE` utilizada no código representa uma matriz de quantização de luminância inspirada no padrão JPEG. Seus menores valores aparecem nas posições associadas às baixas frequências, que concentram a maior parte da informação visual da imagem, enquanto os maiores valores aparecem nas regiões de frequências mais altas, onde normalmente estão detalhes finos e variações menos perceptíveis ao olho humano. Dessa forma, ao dividir os coeficientes da DCT por essa matriz e arredondar os resultados, parte das altas frequências é reduzida ou descartada, produzindo compressão com perda. O parâmetro `FATOR_QUALIDADE` foi usado para intensificar esse efeito: valores maiores aumentam a compressão, mas também podem reduzir a qualidade da imagem reconstruída.

Listing 5: Matriz base de quantização e fator de qualidade usados na compressão.

```

MATRIZ_Q_BASE = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
], dtype=np.float32)

FATOR_QUALIDADE = 2.0

```

Listing 6: Adaptação dinâmica da matriz de quantização.

```

def obter_matriz_q(tamanho_bloco):
    if tamanho_bloco == 8:
        return MATRIZ_Q_BASE * FATOR_QUALIDADE

    if tamanho_bloco % 8 == 0:
        fator = tamanho_bloco // 8
        matriz_expandida = np.kron(
            MATRIZ_Q_BASE,

```



```

        np.ones((fator, fator), dtype=np.float32)
    )
    return matriz_expandida * FATOR_QUALIDADE

return np.ones((tamanho_bloco, tamanho_bloco), dtype=np.float32) * (16 *
FATOR_QUALIDADE)

```

Por fim, a imagem foi ajustada para possuir dimensões múltiplas do tamanho do bloco. Em seguida, cada bloco foi processado separadamente: primeiro pela DCT, depois pela quantização, desquantização e IDCT. A imagem reconstruída foi limitada ao intervalo válido de intensidades, entre 0 e 255.

Listing 7: Processamento da imagem em blocos e reconstrução final.

```

nova_altura = (altura // BLOCK_SIZE) * BLOCK_SIZE
nova_largura = (largura // BLOCK_SIZE) * BLOCK_SIZE
gray = gray[:nova_altura, :nova_largura]
altura, largura = gray.shape

img_reconstruida = np.zeros_like(gray, dtype=float)

for y in range(0, altura, BLOCK_SIZE):
    for x in range(0, largura, BLOCK_SIZE):
        bloco = gray[y:y+BLOCK_SIZE, x:x+BLOCK_SIZE]
        bloco_dct = dct2d(bloco, BLOCK_SIZE)
        bloco_quantizado = np.round(bloco_dct / MATRIZ_Q_AJUSTADA)
        bloco_desquantizado = bloco_quantizado * MATRIZ_Q_AJUSTADA
        bloco_idct = idct2d(bloco_desquantizado, BLOCK_SIZE)
        img_reconstruida[y:y+BLOCK_SIZE, x:x+BLOCK_SIZE] = bloco_idct

imagem_reconstruida = np.clip(img_reconstruida, 0, 255).astype(np.uint8)

```

Resultados

Os resultados da Questão 1 permitiram comparar a imagem original em tons de cinza, a imagem reconstruída após a compressão e a imagem de diferença para diferentes tamanhos de bloco. A reconstrução preservou a estrutura principal da imagem, mas apresentou perdas associadas à etapa de quantização, especialmente em detalhes finos e regiões com variações mais abruptas de intensidade.

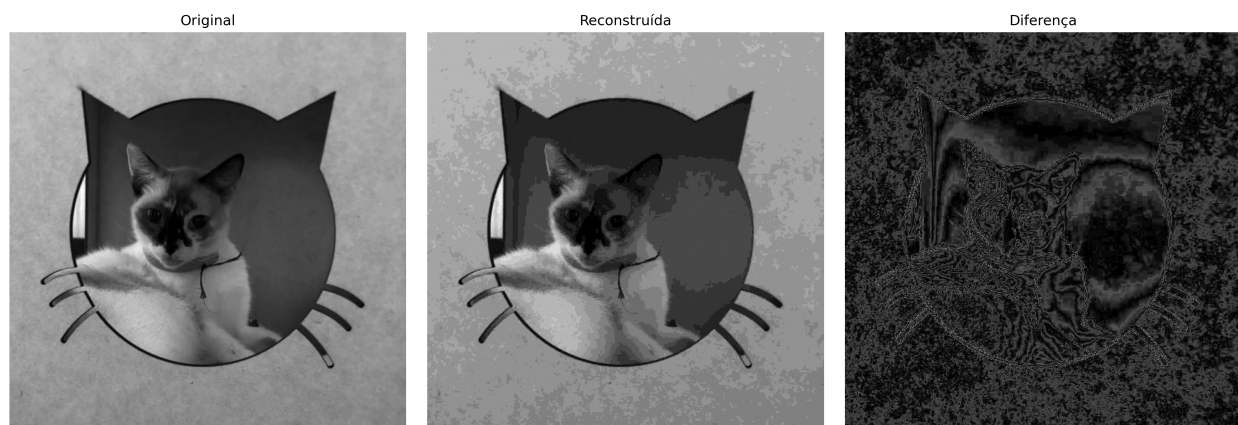


Figura 1: Resultado da compressão com DCT utilizando blocos de 2x2.

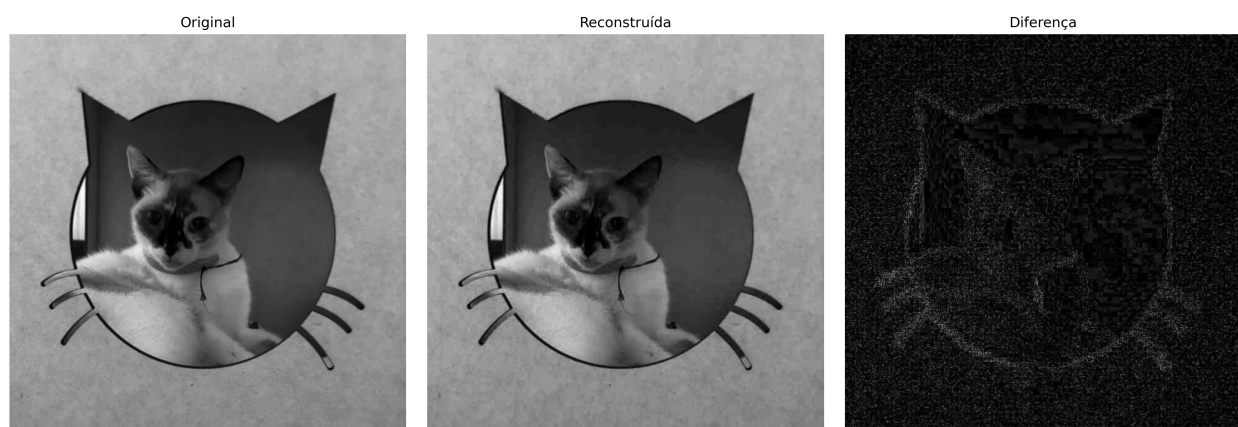


Figura 2: Resultado da compressão com DCT utilizando blocos de 8x8.

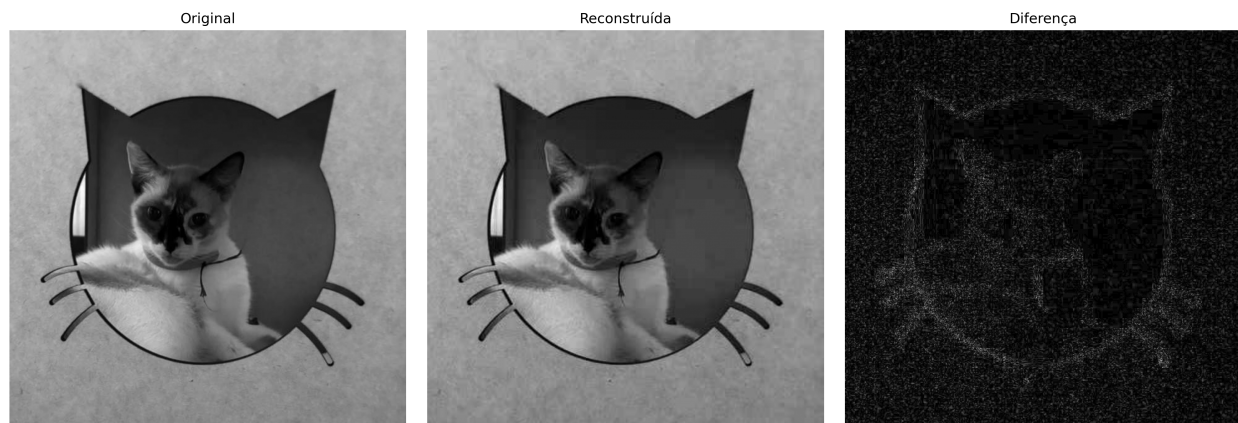


Figura 3: Resultado da compressão com DCT utilizando blocos de 16x16.



Figura 4: Resultado da compressão com DCT utilizando blocos de 64x64.

Análise

A comparação entre a imagem original e a imagem reconstruída mostrou que a compressão com DCT preserva a estrutura geral da cena, mas introduz perdas visuais perceptíveis. Essas perdas ocorrem principalmente durante a quantização, etapa em que parte dos coeficientes de alta frequência é reduzida ou descartada. Como essas frequências estão associadas a detalhes finos, bordas, texturas e pequenas variações de intensidade, a imagem reconstruída tende a apresentar suavização em regiões detalhadas e redução da nitidez em relação à original.

Nos testes com diferentes tamanhos de bloco, também foi possível observar o aparecimento de artefatos de bloco, especialmente quando blocos maiores foram utilizados. Esses artefatos aparecem como divisões ou descontinuidades visuais entre regiões processadas separadamente, pois cada bloco é transformado, quantizado e reconstruído de forma independente. Assim, blocos menores, como 2x2 e 8x8, tendem a acompanhar melhor as variações locais da imagem, enquanto blocos maiores, como 16x16 e principalmente 64x64, podem tornar as perdas mais evidentes, com maior suavização e maior percepção das fronteiras entre blocos.

A compressão com DCT mostrou que grande parte da informação visual pode ser concentrada em coeficientes de baixa frequência. A matriz de quantização reduziu a influência das frequências mais altas, possibilitando a compressão com perda. Nos testes com blocos de diferentes tamanhos, observou-se que a escolha do tamanho do bloco influencia diretamente o resultado: blocos menores tendem a preservar melhor variações locais, enquanto blocos maiores podem aumentar a simplificação da imagem e tornar as perdas mais perceptíveis.

Insights Gerais

- A DCT é eficiente para representar imagens no domínio da frequência.
- A quantização é a etapa responsável pela maior parte da perda visual.
- O tamanho dos blocos influencia a qualidade da reconstrução.
- A comparação com a imagem de diferença facilita a visualização das perdas.

4.2 Questão 2 – Descritores de imagem

Descrição

Na segunda questão, foram implementados descritores simples de imagem com o objetivo de representar quantitativamente características visuais relevantes, como brilho, contraste, uniformidade dos níveis de cinza e variações espaciais de textura. Para isso, duas imagens foram carregadas, convertidas manualmente para escala de cinza e analisadas por meio de métricas estatísticas e estruturais.

Os descritores extraídos foram média, variância, energia, diferença absoluta horizontal e diferença absoluta vertical. A média foi utilizada para representar o brilho geral da imagem; a variância, para indicar o contraste global; a energia, para avaliar a uniformidade da distribuição dos níveis de cinza; e as diferenças horizontais e verticais, para medir variações locais entre pixels vizinhos, associadas à textura e à presença de transições na imagem.

Implementação

O processo foi organizado nas seguintes etapas:

- carregamento de duas imagens para comparação;
- conversão manual das imagens coloridas para escala de cinza;
- cálculo da média dos níveis de intensidade;
- cálculo da variância para estimar o contraste global;
- cálculo da energia a partir do histograma normalizado;
- cálculo das diferenças absolutas horizontal e vertical entre pixels vizinhos;
- organização dos descritores em dicionários para facilitar a comparação;
- geração de gráfico comparativo e salvamento dos resultados em arquivo de texto.

Assim como nas etapas anteriores da atividade, a conversão para escala de cinza foi realizada manualmente por meio da fórmula de luminância. Essa etapa transforma a imagem colorida em uma única matriz de intensidades, permitindo que os descritores sejam calculados diretamente sobre os valores dos pixels.

Listing 8: Conversão manual da imagem colorida para escala de cinza.

```
def to_gray(img):
    altura, largura, _ = img.shape
    gray = np.zeros((altura, largura), dtype=np.uint8)

    for y in range(altura):
        for x in range(largura):
            b = img[y, x, 0]
            g = img[y, x, 1]
```

```

        r = img[y, x, 2]

        valor = int(0.299 * r + 0.587 * g + 0.114 * b)
        if valor > 255:
            valor = 255

        gray[y, x] = valor

    return gray

```

O primeiro descritor calculado foi a média, que representa o brilho geral da imagem. Valores médios mais altos indicam imagens mais claras, enquanto valores menores indicam predominância de regiões escuras.

Listing 9: Cálculo manual da média dos níveis de cinza.

```

def calcular_media(img):
    altura, largura = img.shape
    soma = 0.0

    for y in range(altura):
        for x in range(largura):
            soma += img[y, x]

    media = soma / (altura * largura)
    return media

```

Em seguida, foi calculada a variância, que mede o quanto os valores dos pixels se afastam da média. Assim, imagens com maior contraste ou maior variação de tons tendem a apresentar variância mais elevada.

Listing 10: Cálculo manual da variância como medida de contraste global.

```

def calcular_variancia(img, media):
    altura, largura = img.shape
    soma = 0.0

    for y in range(altura):
        for x in range(largura):
            soma += (img[y, x] - media) ** 2

    variancia = soma / (altura * largura)
    return variancia

```

A energia foi calculada a partir do histograma normalizado da imagem. Esse descritor soma os quadrados das probabilidades de ocorrência dos níveis de cinza, sendo útil para indicar o grau de uniformidade da distribuição tonal. Quando poucos níveis de cinza predominam, a energia tende a ser maior; quando a distribuição é mais espalhada, a energia tende a ser menor.

Listing 11: Cálculo da energia a partir das frequências dos níveis de cinza.

```

def calcular_energia(img):
    altura, largura = img.shape

```

```

frequencias = [0] * 256

for y in range(altura):
    for x in range(largura):
        frequencias[img[y, x]] += 1

energia = 0.0
for i in range(256):
    p_i = frequencias[i] / (altura * largura)
    energia += p_i ** 2

return energia

```

Além das medidas estatísticas globais, também foram calculadas medidas de variação espacial. A diferença absoluta horizontal compara cada pixel com seu vizinho à direita, enquanto a diferença absoluta vertical compara cada pixel com o vizinho da linha seguinte. Essas métricas ajudam a descrever a presença de textura, bordas e mudanças locais de intensidade.

Listing 12: Cálculo das diferenças absolutas horizontal e vertical.

```

def diferenca_horizontal(img):
    altura, largura = img.shape
    diff_horizontal = 0.0

    for y in range(altura):
        for x in range(largura - 1):
            atual = img[y, x]
            vizinho = img[y, x + 1]
            diff_horizontal += abs(atual - vizinho)

    diff_horizontal /= (altura * (largura - 1))
    return diff_horizontal

def diferenca_vertical(img):
    altura, largura = img.shape
    diff_vertical = 0.0

    for y in range(altura - 1):
        for x in range(largura):
            atual = img[y, x]
            vizinho = img[y + 1, x]
            diff_vertical += abs(atual - vizinho)

    diff_vertical /= ((altura - 1) * largura)
    return diff_vertical

```

Os descritores foram reunidos em uma única função, o que tornou o código mais organizado e facilitou a aplicação do mesmo conjunto de medidas em imagens diferentes. A conversão para float foi utilizada para evitar problemas de estouro de valores durante os cálculos matemáticos.

Listing 13: Função responsável por extrair todos os descritores da imagem.

```

def extrair_descritores(img_gray):
    img_f = img_gray.astype(float)

```

```

media = calcular_media(img_f)
variancia = calcular_variancia(img_f, media)
energia = calcular_energia(img_gray)

diff_horizontal = diferenca_horizontal(img_f)
diff_vertical = diferenca_vertical(img_f)

return {
    "Media": media,
    "Variancia": variancia,
    "Energia": energia,
    "Diff_Horizontal": diff_horizontal,
    "Diff_Vertical": diff_vertical
}

```

Por fim, os descritores das duas imagens foram comparados em um gráfico de barras e também salvos em um arquivo .txt. Essa etapa facilitou a análise visual e numérica das diferenças entre as imagens avaliadas.

Listing 14: Organização dos valores e geração do gráfico comparativo dos descritores.

```

descritores = {
    "Media": (descritores1["Media"], descritores2["Media"]),
    "Variancia": (descritores1["Variancia"], descritores2["Variancia"]),
    "Energia": (descritores1["Energia"], descritores2["Energia"]),
    "Dif. Horizontal": (descritores1["Diff_Horizontal"], descritores2["Diff_Horizontal"]),
    "Dif. Vertical": (descritores1["Diff_Vertical"], descritores2["Diff_Vertical"])
}

fig, axs = plt.subplots(2, 3, figsize=(15, 8))
axs = axs.flatten()

for i, (nome, valores) in enumerate(descritores.items()):
    barras = axs[i].bar(["Imagem 1", "Imagem 2"], valores)

    axs[i].set_title(nome)
    axs[i].set_ylabel("Valor")
    axs[i].grid(axis="y", alpha=0.3)

    # margem superior para os textos
    axs[i].set_ylim(0, max(valores) * 1.15)

    # valor acima das barras
    for barra in barras:
        altura = barra.get_height()
        axs[i].text(
            barra.get_x() + barra.get_width() / 2,
            altura,
            f"{altura:.4f}",
            ha="center",
            va="bottom",
            fontsize=8

```

```
)  
  
# remove o sexto subplot vazio  
fig.delaxes(axes[5])  
  
plt.suptitle("Comparacao dos Descritores", fontsize=16)  
plt.tight_layout()  
plt.savefig("resultados/q2/comparacao_descritores.png", dpi=300, bbox_inches="tight")  
plt.show()
```

Resultados

Os resultados da Questão 2 consistiram nos valores numéricos dos descritores extraídos para duas imagens: a Imagem 1, correspondente à aurora boreal, e a Imagem 2, correspondente ao macaco. As duas imagens foram convertidas para escala de cinza antes do cálculo dos descritores, permitindo comparar suas características com base apenas nos níveis de intensidade.



Figura 5: Imagens utilizadas na extração dos descritores após a conversão para escala de cinza: aurora boreal à esquerda e macaco à direita.

Os valores obtidos para cada descritor são apresentados na Tabela 1. Esses dados permitem observar diferenças entre as imagens quanto ao brilho médio, contraste, uniformidade tonal e variações

espaciais nas direções horizontal e vertical.

Tabela 1: Descritores extraídos das imagens analisadas.

Descritor	Imagem 1 – Aurora boreal	Imagem 2 – Macaco
Média	103.4461	120.4846
Variância	1719.6274	3674.0724
Energia	0.0078	0.0055
Diferença horizontal	4.7044	1.8677
Diferença vertical	5.2370	2.2869

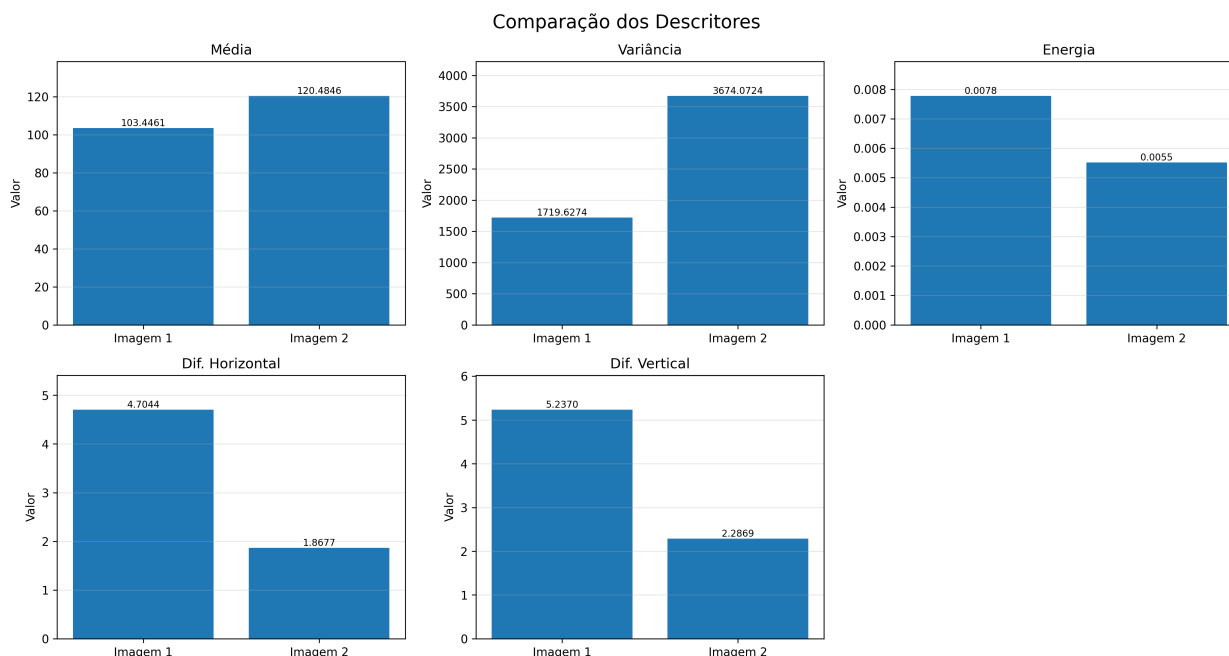


Figura 6: Gráficos comparativos dos descritores extraídos da aurora boreal e da imagem do macaco.

Análise

A comparação entre a aurora boreal e a imagem do macaco mostra como descritores simples podem diferenciar padrões visuais distintos. A média indicou que a Imagem 2 possui maior brilho geral, pois apresentou valor médio de intensidade superior ao da Imagem 1. A variância também foi maior na imagem do macaco, indicando maior contraste global e maior dispersão dos níveis de cinza. Esse comportamento sugere que a segunda imagem possui uma distribuição mais ampla de tons, possivelmente causada pela presença de regiões claras e escuras bem definidas.

Por outro lado, a energia foi maior na imagem da aurora boreal. Isso indica uma distribuição tonal mais concentrada, com maior repetição de determinados níveis de cinza, característica que pode estar associada a regiões mais homogêneas ou com transições suaves de iluminação. A imagem do macaco apresentou menor energia, o que sugere maior diversidade tonal e menor uniformidade na distribuição dos níveis de cinza.

As diferenças absolutas horizontal e vertical foram maiores na imagem da aurora boreal. Esse resultado indica que, embora a imagem do macaco tenha maior contraste global, a aurora apresenta variações locais mais intensas entre pixels vizinhos, especialmente devido às faixas luminosas e às transições de luz presentes na cena. Já os menores valores de diferença horizontal e vertical na imagem do macaco sugerem que, localmente, há transições mais suaves entre pixels vizinhos, mesmo que a imagem apresente maior variação global de intensidade.

Esses descritores podem ser utilizados em tarefas de classificação de imagens porque transformam características visuais em valores numéricos comparáveis. Por exemplo, imagens com alta energia e baixa variância podem ser associadas a regiões mais homogêneas; imagens com alta variância podem indicar maior contraste; e imagens com maiores diferenças horizontais e verticais podem representar texturas, bordas ou padrões com alto nível de detalhe local. Dessa forma, mesmo descritores simples podem servir como atributos de entrada para sistemas de classificação, auxiliando na distinção entre categorias visuais, como paisagens naturais, animais, texturas ou cenas com diferentes níveis de complexidade.

Insights Gerais

- Descritores numéricos permitem comparar imagens de forma objetiva.
- Média e variância resumem características globais importantes.
- Energia ajuda a interpretar a uniformidade dos tons de cinza.
- Diferenças entre pixels vizinhos indicam textura e transições locais.

5 Conclusão

A atividade permitiu consolidar conceitos de compressão, representação e descrição de imagens digitais por meio de implementações práticas. Na primeira questão, a aplicação da DCT evidenciou como a imagem pode ser representada no domínio da frequência e comprimida com perda por meio da quantização. Na segunda questão, os descritores mostraram como características visuais podem ser expressas numericamente, facilitando a comparação entre imagens.

De modo geral, os experimentos reforçaram a importância de escolher métodos adequados conforme o objetivo do processamento, seja para reduzir redundâncias, reconstruir imagens ou descrever seu conteúdo de maneira quantitativa.